SHOULD BE REPLACED ON REQUIRED TITLE PAGE

Instruction

1. Open needed docx template (folder "title"/<your department or bach if bachelor student>.docx).

2. Put Thesis topic, supervisor's and your name in appropriate places on both English and Russian languages.

3. Put current year (last row).

4. Convert it to "title.pdf," replace the existing one in the root folder.

Contents

List of Tables

Abstract

Name capture during expression substitution remains a critical challenge in programming language implementation and formal verification. While the Free-Foil framework resolves this issue and automatically generates type-safe syntax, it currently lacks dedicated tooling for rapid IDE integration. To address this gap, this thesis presents a prototype of an agnostic language server built on Free-Foil that implements core Language Server Protocol (LSP) features: syntax highlighting, diagnostics, hover information, go-to-definition, renaming, and type checking, while requiring little effort from the language author. Each feature is enabled by implementing a small set of type classes over the language's signature and binders, which keeps tree traversal and scope management inside a reusable library. A worst-case analysis shows that every server algorithm runs in at most linear time relative to the syntax tree size. The work also identifies the limitations of the current implementation and directions for extending it. The prototype is well-suited for educational projects, language prototyping, and rapid integration of domain-specific languages into modern IDEs, with its source code and documentation providing a foundation for further research in automated development tooling.

Chapter 1

Introduction

Language servers are an essential component of modern development environments, providing rich code intelligence features such as code completion, navigation, error checking, and refactoring support. The Language Server Protocol (LSP) [1], introduced by Microsoft, has standardized communication between language servers and editors, allowing a common interface for diverse programming environments. Implementing a language server from scratch entails a great deal of boilerplate code and similar, repetitive algorithms for every feature, which makes the work slow; more code also tends to mean more bugs that are harder to identify later. This thesis aims to make the production of a minimal language server a simpler task by moving as much processing as possible into an agnostic framework built on safe and predictable machinery. It presents a Haskell framework that, given a Free Foil abstract syntax and a set of functions depending on the language-specific signature, produces a language server communicating via LSP that can store the current environment's configuration, update it on file changes, rename symbols, navigate to a symbol's definition, perform type checking, and present informational, diagnostic, and warning messages.

1.1 Binding and Scopes

One of the most persistent aspects of language server implementation, as in compiler implementation, is the correct management of variable bindings and scopes. In the following Kotlin example, x cannot simply be replaced with y in a's body when calling a(y), because that would mix the local variable x with the free y.

```
val y = 1
fun a (x: Int): (Int) -> Int = { y -> x + y }
a(y)
```

Handling bound identifiers safely and efficiently is a long-studied problem, with approaches ranging from De Bruijn indices [2] through locally nameless and nominal techniques to intrinsically scoped representations; Chapter 2 surveys this design space in detail. This thesis builds on Free Foil [3], which combines free scoped monads [4] with the type-safe, single-pass approach of Foil [5]. Free Foil was chosen because, besides being safe and efficient, it generalizes the Abstract Syntax Tree (AST) over a declarative bifunctor signature, so that features such as navigation to a symbol's definition, renaming, and the reporting of scope-related errors can be implemented once, agnostically, and reused for any language with minimal effort.

Free Foil parameterizes the abstract syntax over a binder type and a signature sig that distinguishes regular nodes from those introducing new scopes, indexing every entity by a phantom scope type S so that the Haskell type system statically rejects operations mixing incompatible scopes. Capture-avoiding substitution and normalization thus become available out of the box, with no manual renaming or index shifting during tree traversal. Crucially, because the signature is a bifunctor

it can implement Bifoldable, enabling a universal recursive traversal of any AST regardless of the specific language. By requiring additional type classes only when specific signature data is needed, IDE features can be implemented as generic algorithms, turning language server development from a repetitive, language-specific task into a configuration-driven process: a developer provides only a declarative signature and a minimal set of type class instances, while the framework handles scope management, tree traversal, and LSP communication.

The ambition of describing binding once and deriving editor services generically is not new. The most developed prior effort is the line of work on scope graphs and the Spoofax/Statix language workbench [6], [7], [8], which specifies name resolution and type checking declaratively and answers them through a runtime search over an extrinsic graph. This thesis instead carries the intrinsic, compiler-checked route of Free Foil across the LSP boundary, trading some of the binding breadth of scope graphs for zero-cost, statically verified scope safety. Chapter 2 develops this comparison (Section 2.2.3), and Chapter 6 returns to it.

1.2 Contribution

This thesis presents a novel, architecture-driven framework for automating the development of language servers by leveraging the Free Foil library. The primary contributions are structured around three core aspects: architectural abstraction, developer ergonomics, and computational guarantees.

First, it introduces a modular, language-agnostic architecture that decouples LSP feature implementation from language-specific syntax. By building directly on Free Foil's type-safe scope management and capture-avoiding substitution, the framework abstracts away the repetitive, error-prone logic traditionally required

for tree traversal, variable tracking, and scope resolution. This enables a single codebase to serve as an agnostic language server capable of handling any language defined via a declarative bifunctor signature.

Second, it defines a lightweight, type-class-driven interface that activates core IDE functionalities with minimal developer overhead. Through a carefully designed set of type classes—RangedSig, FoldablePat, TypeDeductiveSig, TypedSig, TypedPat, HoverableSig, HoverablePat, TokenizableSig, TokenizablePat, DiagnosableSig, DiagnosablePat—the framework exposes syntax highlighting, diagnostics, go-to-definition, symbol renaming, hover tooltips, and basic type checking. A language integrator implements only a declarative AST signature and a small set of type class instances: 14 type-class methods together with the buildAsts function, 15 functions in total. All scope management, recursive traversal, and LSP communication are handled automatically by the library, shifting the development paradigm from boilerplate-heavy implementation to configuration-driven setup.

Third, it provides a formal worst-case complexity analysis of all server algorithms, demonstrating that every operation runs in at most linear time relative to the AST size ($O(K)$); position-based queries such as go-to-definition and hover descend a single path and, for fixed-arity grammars, run in time linear in the tree depth ($O(L)$, with $L \leq K$). This guarantees predictable performance and scalability for typical development workflows.

The framework is validated by two independent example language servers built on it without modifying any core module: LambdaPi, a dependently typed lambda calculus with Π-types, and Flan, a functional language with let-bindings, pair patterns, and conditionals that exercises the full feature set. From a practical standpoint, the proposed framework significantly accelerates the creation of

language servers, making it particularly valuable for educational projects, rapid language prototyping, and the integration of domain-specific languages into modern IDEs. The thesis also critically evaluates the current implementation, identifying key limitations such as the absence of global namespace resolution, reliance on simple annotation-based type inference (lacking constraint unification), and bulk cache updates upon file modification. In response, it outlines concrete directions for future work, including incremental type checking, interval-map indexing for logarithmic-time ($O(\log K)$) position lookups, single-pass tree traversals, and generic default implementations for common LSP features. The source code and documentation are provided to serve as a foundational toolkit for further research in automated development tooling.

1.3 Thesis Outline

The remainder of this thesis is organized as follows. Chapter 2 reviews the techniques for safe and efficient handling of bound variables, the existing language-agnostic models of name binding and resolution, and the Language Server Protocol. Chapter 3 presents the methodology: the design requirements, the agnostic tree-traversal strategy, and the algorithms behind each supported feature. Chapter 4 documents the implementation, covering the LSP interface, caching, name-collision handling, and the Flan example integration. Chapter 5 evaluates the framework against its design criteria and analyzes its automation and computational complexity, together with current limitations and directions for future work. Chapter 6 concludes.

Chapter 2

Literature Review

2.1 Name Capture

The problem of name capture is fundamental in programming language implementation, theorem proving, and formal verification systems. When substituting expressions containing bound variables, naïve replacement can lead to variable capture, where a free variable in the substituted term becomes bound, changing the meaning of the expression. For example, in lambda calculus, substituting y for x in $(\lambda \mathrm{y}.\mathrm{x})$ would incorrectly yield $(\lambda \mathrm{y}.\mathrm{y})$ if performed naïvely. Various approaches to preventing this issue have been developed, each with trade-offs between safety, efficiency, and expressiveness.

The simplest approach, often called "naïve substitution," [9] uses explicit variable names and requires careful renaming during substitution to avoid capture. While intuitive, this approach is error-prone and inefficient as it requires maintaining and checking against a global context of variable names. De Bruijn indices [2] offer an alternative by replacing names with numeric indices representing binding distance. This solves name capture but creates other issues: terms become difficult

to read, and substitution still requires traversing the entire abstract syntax tree (AST) to adjust indices.

Higher-Order Abstract Syntax (HOAS) [10] leverages the host language's binding mechanisms, providing excellent safety but often at the cost of expressiveness when implementing algorithms that need to inspect or transform bound variables. Variants of HOAS address some of these limitations: Parametric HOAS [11] threads a type parameter through every binding position, ruling out exotic terms while keeping the approach implementable in a conventional type system, and the boxes-go-bananas encoding [12] similarly uses parametric polymorphism to achieve a sound embedding. Nevertheless, both variants retain the core HOAS difficulty of working under binders. The "rapier" technique used in the Glasgow Haskell Compiler [13] offers better efficiency by maintaining a stateless local scope and performing capture-avoiding substitution in one pass. However, as noted by Maclaurin et al. [5], it lacks type safety, making it easy to forget scope extensions or necessary renamings when implementing complex language features.

A separate family, the nominal techniques, treats names themselves as the primitive notion. Building on the nominal sets of Gabbay and Pitts [14], systems such as FreshML [15] and Nominal Isabelle [16] provide first-class support for binding, fresh-name generation, and reasoning modulo α-equivalence, mainly in functional programming and proof-assistant settings. They offer strong metatheoretic guarantees but enforce freshness dynamically or through dedicated logical infrastructure rather than through the host language's type system, as the intrinsically typed representations below do.

2.1.1 Intrinsic Scoping

Intrinsic scoping addresses the safety issues of previous approaches by encoding scope information directly in types. With this technique, the type system ensures that all terms are well-scoped by construction, making ill-scoped terms impossible to express. Bird and Paterson's approach [17] uses nested data types to represent De Bruijn indices at the type level. For Haskell:

```
data Exp a
  = Var a
  | App (Exp a) (Exp a)
  | Lam (Exp (Maybe a))
```

Here, the type parameter changes when going under a binder (using Maybe), enforcing proper scope extension. While this guarantees safety, it still requires traversing and updating all variables during substitution, impacting efficiency. Additionally, the deep polymorphic recursion consumes significant memory and complicates implementing algorithms that work under binders.

Kmett's bound library [1] extends this approach with more flexible representations like locally nameless [18] and co-de-Bruijn indices [19]. These variations improve ergonomics but still face fundamental efficiency limitations due to their reliance on explicit index manipulation.

2.1.2 The Foil

The Foil [5] resolves the efficiency-safety trade-off by combining the stateless, single-pass algorithm of the rapier with strong type-level guarantees. It

[1] https://hackage.haskell.org/package/bound

implements the Barendregt convention [20], claiming all bound variables to be distinct and different from free variables, with type safety. The Foil achieves this by indexing all expressions, names, and scopes with phantom a type parameter S that represents the current set of in-scope variables. Using phantom types to carry compile-time evidence without runtime cost is a technique known as "ghosts of departed proofs" [21], and the Foil is a systematic application of it to scope safety. S is defined as `data S = VoidS`

So every entity in the Foil's representation is scope-indexed by a kind S:

- `newtype Name (n :: S)` is the type of a name that belongs to the scope `n`. The name's underlying representation is a machine-sized integer.
- `newtype Scope (n :: S)` is the type of a scope that contains exactly names of type `Name n`. Similarly to the rapier, it stores an integer map [22] to lookup and extend the scope.
  - if `scope1 :: Scope n` and `scope2 :: Scope n`,
    then `scope1 == scope2`
  - if `scope :: Scope n` and `x : Name n`,
    then `x `member` scope == True`.
- `newtype NameBinder (n :: S) (l :: S)` is the type of a binder that introduces a single new name. It extends the scope indexed by `n` into the scope indexed by `l` with the name contained in it. New instances of `NameBinder n l` can only be created with a `withFresh` function, which introduces new instances of S-indexed types with rank-2 polymorphism:

```
withFresh :: Distinct n
  => Scope n
```

```
  -> (forall l. (Distinct l, Ext n l) => NameBinder n l -> r)
  -> r
```

With these data types, a simply-typed lambda calculus can be represented as the following:

```
data Expr n where
  Var :: Name n -> Expr n
  App :: Expr n -> Expr n -> Expr n
  Lam :: NameBinder n l -> Expr l -> Expr n
```

Here, every expression is indexed with a scope `s`, as well as variables, which are represented as `Name n`. It is noteworthy that the body of the lambda expression is indexed with a different scope `l`, which is exactly the scope `NameBinder n l` creates. Thus, when going under a binder, the type of the sub-expression changes, imposing a type-level constraint on all the terms within it.

When implementing substitution, the type system ensures:

- Substitution is applied to all sub-terms
- Scope is properly extended when entering binders
- Variables are correctly renamed to avoid capture

The Foil's efficiency comes from its `Sinkable` type class, which allows "sinking" expressions into extended scopes:

```
sink :: (Sinkable e, Distinct n, Ext n l) => e n -> e l
sink = unsafeCoerce
```

Since the Foil uses name-based representation rather than indices, and the underlying representation of names remains unchanged between scopes, this operation is a type-safe zero-cost coercion. This contrasts with De Bruijn-based approaches that must traverse and increment every index.

Despite its advantages, the Foil requires manually writing syntax representations and substitution implementations for each language, creating boilerplate code and limiting reusability across different languages.

2.1.3 Free Foil

Free Foil bridges the gap between generic syntax representation and efficient, type-safe substitution. It combines Kudasov's free scoped monads [4] with the Foil to automatically generate well-scoped syntax with capture-avoiding substitution from a language's second-order signature, a notion whose equational theory and formal metatheory are developed in [23], [24]. The core idea is to parameterize abstract syntax additionally by a bifunctorial signature sig that distinguishes between regular sub-terms and scoped sub-terms (those that may access newly bound variables) and, additionally, by the binder:

```
data ScopedAST binder sig n where
  ScopedAST :: binder n l -> AST binder sig l -> ScopedAST binder sig n
data AST binder sig n where
  Var :: Name n -> AST binder sig n
  Node
    :: sig (ScopedAST binder sig n) (AST binder sig n)
    -> AST binder sig n
```

Given a bifunctorial signature sig, the AST data type automatically generates

the complete syntax with proper scoping. For simply-typed lambda calculus:

```
data ExprF scope term
  = AppF term term
  | LamF scope
type Expr = AST NameBinder ExprF
```

Because sig is an abstract type parameter, this representation also supports the data-types-à-la-carte approach [25]: a coproduct of two signatures can be passed as sig, allowing independent language features to be composed rather than defined in a single monolithic type.

CoSinkable typeclass introduces coSinkabilityProof to support patterns. Given a renaming function and a pattern, it provides a renaming function for the variables bound by the pattern, and a refreshed version of the pattern, hence, making sure variables in patterns are refreshed properly to avoid name capture.

The key innovation is that Free Foil inherits the Foil's efficient substitution algorithm while maintaining complete type safety. The Sinkable instance for AST enables zero-cost scope extensions, and capture-avoiding substitution is derived generically:

```
newtype Substitution (e :: S -> Type) (i :: S) (o :: S)
  = UnsafeSubstitution (IntMap (e o))
substitute :: (Bifunctor sig, Distinct o, CoSinkable binder, SinkableK binder)
  => Scope o
  -> Substitution (AST binder sig) i o
  -> AST binder sig i
  -> AST binder sig o
```

Unlike free scoped monads that use nested De Bruijn indices, Free Foil eliminates the need for explicit index manipulation, dramatically improving performance while maintaining the same level of type safety. The `substitute` and `Var` operations are reminiscent of the bind and unit of a monad; formally, the type `AST' sig n = Scope n -> AST binder sig n` can be given the structure of a relative monad [26], a generalization of monads to functors that do not necessarily map a category to itself. Benchmarks show that while Free Foil is slightly slower than direct Foil implementations due to the additional layer of abstraction [3], it outperforms nested De Bruijn approaches by orders of magnitude for complex terms.

Additionally, Free Foil supports a template-based approach where scope-safe representations can be automatically derived from naïve AST definitions using Template Haskell [27], allowing seamless integration with parser generators like BNF Converter (BNFC) [28].

Because a Free Foil signature is an ordinary bifunctor, traversals over the resulting syntax can reuse standard datatype-generic programming techniques instead of bespoke recursion. Generic traversal has a long lineage in Haskell, from recursion schemes over the fixed point of a functor [29] to Scrap Your Boilerplate [30]. Free Foil fits this tradition: deriving `Bifunctor` and `Bifoldable` for a signature suffices to fold over every node of any language's syntax, which is exactly the property the IDE features in this thesis rely on to stay language-agnostic.

2.2 Language-Agnostic IDE Tooling

The techniques surveyed so far address name capture from the perspective of representing and manipulating terms inside a single language implementation. A complementary line of research asks a broader question: can name binding and

resolution be described once, in a language-independent way, so that editor services such as go-to-definition, code completion, and rename refactoring are derived generically rather than re-implemented for every language? This is precisely the ambition shared by this thesis, and the most developed answer to date is the line of work on scope graphs by the Delft programming languages group, which underpins the Spoofax language workbench [8].

2.2.1 Scope Graphs: A Theory of Name Resolution

Neron, Tolmach, Visser, and Wachsmuth [6] observe that name resolution "cuts across the local inductive structure of programs": the declaration introduced by a let node may be referenced by an arbitrarily distant descendant, and qualified or imported names escape lexical nesting entirely. Defining resolution directly over the abstract syntax tree therefore leads to ad-hoc, per-language symbol-table code that is duplicated across the compiler, the IDE, and any mechanized semantics. Their proposal is to factor name resolution into two stages: a language-specific construction of a language-independent scope graph from the AST, followed by a language-independent resolution process over that graph.

A scope graph collapses all AST nodes that "behave uniformly with respect to name resolution" into nodes called scopes. Each scope carries a set of declarations (binding occurrences x_i^D, indexed by their AST position i) and references (applied occurrences x_i^R). Edges between scopes encode visibility: a parent edge P models lexical nesting, while an import edge models the inclusion of a named scope's contents (used to model ML-style modules, and, elegantly, Java class inheritance as "the $\mathrm{extends}$ clause playing the role of import"). Resolution is then specified declaratively as a resolution calculus that searches for a path from a reference to a declaration. Shadowing, where an inner declaration hides an outer one, is captured

not by deleting candidates but by a specificity ordering on paths: a path with fewer parent steps is more specific, local declarations are preferred over imports, and imports over lexical parents. A reference resolves to a declaration only if that declaration is visible, that is, reachable via a path that no other resolution shadows. The authors give a deterministic, terminating resolution algorithm built from an environment-shadowing operator and prove it sound and complete with respect to the calculus.

Crucially for the goals of this thesis, scope graphs are explicitly designed to support front-end tooling rather than only well-formed programs. The calculus deliberately admits ambiguous resolutions (multiple declarations for one reference) and unresolved references (free variables), because "such programs are the normal case in IDEs and other front-end tools." On top of resolution, the authors derive language-independent notions of α-equivalence, in which two programs are α-equivalent if they are structurally identical up to identifiers and induce the same equivalence classes of positions, and of valid renaming as a renaming that preserves α-equivalence, that is, introduces no capture. This is exactly the rename-refactoring service an IDE must provide, obtained here as a corollary of the resolution theory rather than as bespoke per-language code.

2.2.2 Scopes as Types: From Resolution to Type Checking

The original scope-graph framework resolves names but says nothing about types, and it was limited to simple, nominal type systems where a type is identified by a name. Van Antwerpen, Bach Poulsen, Rouvoet, and Visser [7] extend the framework with the insight that a type can itself be modeled as a scope. A record type, a class, or a parameterized type all bind names internally; by representing such a type as a scope in the graph, the internal structure of types is expressed

with the very same generic machinery used for ordinary lexical binding. Structural subtyping becomes a visibility edge between scopes, and instantiating a generic type becomes the creation of a fresh scope that refines a type parameter's binding.

To make these specifications executable as type checkers, the authors generalize resolution in two ways and embed it in a constraint language. First, plain references are generalized to scope graph queries parameterized by a regular-expression label well-formedness predicate and a per-query visibility ordering, so that different namespaces can obey different visibility policies. Second, they introduce Statix, a declarative meta-language in which a language designer writes guarded inference-rule-style constraints that simultaneously construct the scope graph and query it, with unification handling type terms. The key engineering contribution is decoupling traversal order from solving order: because constraints are solved by a generic solver rather than during a fixed AST traversal, the language designer is freed from manually staging passes to "collect information before it is needed" (for example, binding all members of a recursive group before checking their bodies). The solver guarantees sound resolution over the gradually-built graph by tracking which scopes may still be extended and delaying any query that could traverse an incomplete scope. The approach is implemented in Spoofax and validated on the simply-typed lambda calculus with records, System F, and Featherweight Generic Java, yielding editors with type checking from a single declarative specification.

2.2.3 Comparison with Free Foil

Scope graphs and Free Foil pursue the same overarching goal, a single, language-independent treatment of binding from which tooling can be derived, but they sit at opposite ends of a design axis, and this contrast motivates the present

work.

Where correctness is enforced. Free Foil encodes scope in the host language's type system: every name, scope, and binder is indexed by a phantom kind $\mathbb{S}$, and the Haskell compiler statically rejects any operation mixing incompatible scopes. Ill-scoped terms are unrepresentable, and capture-avoiding substitution is correct by construction. Scope graphs, by contrast, use an extrinsic model: the graph is an ordinary data structure built alongside the AST, and resolution is a runtime search whose correctness is established by meta-theoretic proof (soundness and completeness of the algorithm with respect to the calculus) rather than by the type system of the implementation language. A bug in the AST-to-graph mapping yields wrong resolutions silently; in Free Foil an analogous mistake is typically a compile-time type error.

Expressiveness of binding. This is where scope graphs are clearly ahead. The scope-graph calculus natively handles non-lexical binding, including modules, qualified names, transitive and cyclic imports, and class inheritance, through import edges and tunable well-formedness and visibility policies [6], and "Scopes as Types" [7] pushes this all the way to structural and parametric type systems. Free Foil, building on the Foil [5] and free scoped monads [4], is designed around lexical binding: a binder extends the scope of the subtree it dominates. Non-lexical structures are not thereby excluded, however. Because $\mathbb{S}$ is only a phantom tag on scopes and the binder is an ordinary type parameter, a scope is an unordered set of names by construction, and a global namespace or an import can be encoded indirectly, as a synthetic binder that extends the scope with the relevant names, with a qualified import reducing to a single-name binder. What scope graphs provide and Free Foil does not is the resolution machinery itself: import edges,

visibility and well-formedness policies, and cross-module path-finding are derived automatically there, whereas here they must be implemented by hand, and cross-module name identity must be tracked manually because names are scope-local and can be transported only along a scope extension. Such manual encodings also fall partly outside Free Foil's by-construction guarantees: the compiler still rejects mixing incompatible scopes, but it does not verify that a synthetic binder faithfully models the intended visibility. The gap is thus one of prepacked expressiveness and ergonomics rather than of what the $\mathbb{S}$-indexed representation can ultimately encode.

Treatment of erroneous and partial programs. Scope graphs are explicitly engineered for the IDE setting: ambiguous and unresolved references are first-class, which is essential for editor services over the incomplete, frequently-broken programs typical of live editing [6]. Free Foil's intrinsic guarantees assume a well-scoped term; mapping a partially-written or ill-scoped source file into an $\mathbb{S}$-indexed AST requires additional care at the boundary, an issue the implementation chapters of this thesis address directly.

Derived services. Both frameworks deliver the same headline IDE features generically. Scope graphs derive go-to-definition (path to a declaration), rename (validity defined via α-equivalence), and, via Statix, type checking, all from a declarative specification interpreted by a generic solver. Free Foil derives capture-avoiding substitution, normalization, and, because a Free Foil signature is a $\mathrm{Bifunctor}$ and hence $\mathrm{Bifoldable}$, generic recursive traversal, from a declarative bifunctor signature compiled by the Haskell type checker. In other words, Spoofax and Statix achieve genericity through runtime constraint solving over an untyped graph, whereas Free Foil achieves it through compile-time type-level guarantees over a scoped AST.

Positioning of this thesis. The two approaches are complementary rather than competing. Scope graphs demonstrate the breadth of binding patterns a truly language-agnostic tool must eventually cover and set the standard for IDE-oriented robustness, but they obtain their guarantees through proof and runtime search and are tied to the Spoofax ecosystem. This thesis instead asks how far the intrinsic, type-safe route of Free Foil can be carried into a standalone, LSP-based language server: trading some of the breadth of scope graphs (notably non-lexical binding and rich type systems) for zero-cost, compiler-checked scope safety and a configuration-driven implementation that lives in mainstream Haskell tooling rather than a dedicated workbench.

2.2.4 Language Workbenches

Spoofax is one of a broader class of language workbenches, tools built on the premise of "define a language, get an IDE for free." Systems such as Xtext [2], JetBrains MPS [3], and Rascal [4] let a designer specify a language's grammar, name binding, and semantics, and generate editor support from that specification. The framework of this thesis shares this philosophy but is deliberately narrower: rather than a full workbench with its own meta-languages and environment, it targets the single goal of deriving a standalone LSP server from a Free Foil signature, and operates within ordinary Haskell tooling.

[2] https://eclipse.dev/Xtext/
[3] https://www.jetbrains.com/mps/
[4] https://www.rascal-mpl.org/

2.2.5 Tree-sitter

A different route to language-agnostic tooling is taken by Tree-sitter [5], an incremental parsing library that re-parses fast enough for every keystroke and recovers gracefully from syntax errors. From a single declarative grammar it powers syntax highlighting and structural navigation across many editors and dozens of languages. Tree-sitter is, however, purely syntactic: it has no notion of name binding, scope, or types, so semantic services such as go-to-definition or rename must still be built on top of it per language. It is thus complementary to the present work, which is concerned precisely with the scope-aware layer that Tree-sitter leaves out.

2.3 Language Server Protocol (LSP)

The Language Server Protocol (LSP) [1] standardizes communication between a language server, a process that performs language analysis, and a language client in the editor. This section reviews what the protocol defines, focusing on the features implemented by the framework of this thesis, and surveys the libraries used to build servers.

2.3.1 Protocol Foundations

LSP is built on JSON-RPC: every message is a JSON object exchanged over a transport such as standard input/output or a socket. The protocol distinguishes three kinds of messages. Requests carry an identifier and expect a matching response, whereas notifications carry no identifier and are fire-and-forget. Communication is

[5] https://tree-sitter.github.io/tree-sitter/

bidirectional: both client and server may initiate requests and notifications, which is what allows a server to push diagnostics to the editor without being asked.

A session begins with a capability negotiation. The client opens with an initialize request advertising the features it supports; the server replies with its own server capabilities, declaring which requests it can answer and configuring options such as how documents should be synchronized. Only after the client acknowledges with an initialized notification does normal operation begin. This handshake means a server need not implement the entire specification: it offers a subset of features, and the editor adapts to whatever is advertised. A symmetric shutdown/exit exchange ends the session.

2.3.2 Text Document Synchronization

A language server does not, in general, read source files from disk. Instead the client owns the authoritative, possibly-unsaved contents of every open document and streams changes to the server. Three notifications govern this lifecycle: textDocument/didOpen announces a newly opened document together with its full text, textDocument/didChange reports edits, and textDocument/didClose signals that the client no longer tracks it. Documents are identified by URIs and carry a monotonically increasing version number so that edits can be ordered unambiguously.

The server chooses, during capability negotiation, how much of a document each change notification carries. With full synchronization the client resends the entire document on every edit, whereas incremental synchronization sends only the changed ranges, leaving the server to maintain the document state. Full synchronization is simpler to implement at the cost of reprocessing the whole file on each keystroke; this trade-off is directly relevant to the caching and re-analysis

strategy discussed in the implementation chapters.

2.3.3 Language Features

Positions and ranges. Almost every language feature is phrased in terms of source locations. LSP encodes a position as a zero-based $(\mathrm{line},\ \mathrm{character})$ pair and a range as a start–end pair of positions. Because these offsets are zero-based and counted in UTF-16 code units rather than Unicode characters, a server must translate between its internal representation and the protocol's convention; parsers that report one-based lines and columns, as is common, require an off-by-one adjustment at the boundary. Mapping each abstract syntax tree node to the range of source text it spans is therefore a prerequisite for almost all the features below.

Navigation and editing. The $\mathrm{textDocument}/\mathrm{definition}$ request asks the server to resolve the symbol under the cursor to the location of its declaration, the canonical "go-to-definition" service. The $\mathrm{textDocument}/\mathrm{rename}$ request asks for all edits needed to consistently rename a symbol throughout the document; the server returns a workspace edit describing the textual changes. Both of these are fundamentally name-resolution queries, which is precisely where intrinsically scoped representations have something to offer.

Informational features. The $\mathrm{textDocument}/\mathrm{hover}$ request returns documentation or type information to display when the cursor rests on a symbol. The $\mathrm{textDocument}/\mathrm{semanticTokens}$ request provides token-level classification for semantic syntax highlighting, allowing the editor to color identifiers according to whether they denote, say, a variable, a function, or a type, which purely lexical highlighting cannot recover. Tokens are returned in a compact, delta-encoded

integer format relative to the previous token.

Diagnostics. Errors and warnings are delivered through textDocument/publishDiagnostics, a server-initiated notification: rather than waiting to be queried, the server pushes a list of diagnostics, each consisting of a range, a severity, and a message, to the client whenever its analysis of a document changes. This is the channel through which scope errors, type mismatches, and unbound symbols surface to the user.

2.3.4 Libraries for Building Language Servers

Because the protocol is verbose and its message types numerous, most servers are built on a library that handles the JSON-RPC framing, message (de)serialization, and lifecycle bookkeeping, leaving the author to supply only the language-specific logic. Eclipse LSP4J [6] provides this foundation on the JVM and underpins Scala's Metals [7] and the Kotlin Language Server [8]. For Haskell, the haskell/lsp [9] family (including lsp-types and lsp-test) offers type-safe representations of the LSP message types together with the communication machinery, and has been hardened through its use in the Haskell Language Server (HLS). The framework presented in this thesis is built on haskell/lsp, delegating protocol handling to it and contributing the layer above: the generic, Free-Foil-based analysis that answers the requests described in Section 2.3.3.

Crucially, these libraries are protocol frameworks, not semantic ones. They marshal messages and manage document state, but they impose no model of abstract

[6] https://github.com/eclipse-lsp4j/lsp4j
[7] https://github.com/scalameta/metals
[8] https://github.com/Kotlin/kotlin-lsp
[9] https://github.com/haskell/lsp

syntax, name binding, or scope; the language author is left to implement resolution, renaming, and type checking from scratch for every language. This is the gap the present work targets: by layering a scope-safe, language-agnostic analysis over a protocol library, the repetitive semantic core of a server is provided once rather than rewritten per language.

2.4 Conclusion

This review examined three threads that together frame the problem this thesis addresses: techniques for safe and efficient handling of bound variables, language-agnostic models of name binding and resolution, and the Language Server Protocol that connects analysis to the editor.

The first thread shows a steady progression from error-prone manual renaming, through De Bruijn indices and nominal techniques, to intrinsically scoped representations. The Foil enforces scope correctness at the type level while retaining the single-pass efficiency of the rapier, and Free Foil generalizes this guarantee to any language described by a bifunctor signature, deriving capture-avoiding substitution and generic traversal without per-language boilerplate. This makes type-safe scoping not merely correct but reusable.

The second and third threads reveal where that reusability has not yet been applied. Scope graphs demonstrate that name resolution, α-equivalence, and renaming can be specified once and reused across languages, but they obtain their guarantees through meta-theoretic proof and runtime constraint solving rather than the host type system, and the most complete realization lives inside the Spoofax workbench. The Language Server Protocol, meanwhile, standardizes how such services reach any editor, and a mature ecosystem of libraries handles its message-

level concerns, yet these libraries stop at the protocol boundary and impose no model of binding or scope, leaving each language to reimplement resolution, renaming, and type checking from scratch.

A gap therefore remains between the two halves of the picture. The machinery for intrinsically scope-safe, language-agnostic syntax exists in Free Foil, and a standard transport for editor services exists in LSP, but no work connects them: language servers are still typically built on extrinsically scoped ASTs with bespoke per-language logic, vulnerable to subtle binding bugs and to repeated implementation effort. Carrying Free Foil's compile-time scope safety across the protocol boundary into a single, configuration-driven server is the opportunity this thesis pursues, trading the broader binding expressiveness of scope graphs for zero-cost, compiler-checked guarantees and a substantially smaller implementation burden per language.

Chapter 3

Methodology

3.1 Design Requirements

The goal of this work is to develop an out-of-box solution for a minimal language server. Given a Foil AST and some signature-dependant functions, it has to compile into a binary communicating via LSP. The work also provides a small example project demonstrating the integration of such a language server into the Visual Studio Code (VS Code) [1], an open-source code editor.

The main criterion is the simplicity for an end user. This also means requiring as less boilerplate code as possible. So that a person wishing to create a proof assistant or a simple programming language for educational or research purposes could have provided the minimal set of functions, while all the heavy-lifting would have lied on the agnostic languages server's shoulders. Specifically, the implementation of LSP methods for symbols definition, renaming, type-checking, code highlighting, producing diagnostic and hover messages. All of the methods above could be implemented by juggling a finite amount of functions.

[1] https://code.visualstudio.com/

3.1.1 Deprioritized Criteria

The project does not focus on substituting complex enterprise-level language servers, nor does it provide a "swiss knife" for any level of complexity projects. Hence, the performance and extensibility are deprioritized. Though, this doesn't mean the behavior of a resulting language server should be non-customizable and slow. Rather it only implies the performance and extensibility must not come in contradiction with the prioritized criteria.

3.1.2 Evaluation Criteria

To assess whether the proposed language server framework meets its design goals, the following criteria will be used:

1. **Feature Completeness**: The framework must support essential LSP features including *go-to-definition*, *symbol renaming*, *showing hover and diagnostic messages*, *type checking*, and *syntax highlighting*. Each feature should function correctly when the corresponding user-provided functions are implemented.

2. **Minimal Entry Threshold**: The framework must provide dummy implementations for the type classes' functions where possible to minimize time required for the first (featureless) language server launch.

3. **Correctness**: Symbol resolution must accurately trace variable definitions to their usages within the correct scope.

4. **Language Agnosticism**: The framework should work with at least two distinct language implementations without modification to the core server

logic. This demonstrates the binder and sig parameterization achieves true abstraction.

5. **Worst-Case Complexity**: While performance is deprioritized, every server operation must run in at most linear time in the size of the syntax tree ($O(K)$). A linear bound is unavoidable for whole-file operations such as syntax highlighting and diagnostics, which must visit every node to produce their output; requiring it of every operation ensures the library introduces no super-linear behavior of its own, while leaving room for individual operations to do better.

6. **VS Code Integration**: The example project must demonstrate successful integration with Visual Studio Code, with all supported LSP features accessible through the editor's standard interface.

These criteria will be evaluated through the implementation of the example project mentioned in Section 3.

3.2 Language Server Methods

3.2.1 Building a Tree

The first operation required for any language server is to build the AST, since it is essentially what is going to be analyzed throughout the course of a user's session. Given a language's file extension, finding certain files, converting them into ASTs, and even tracking modifications is relatively easy. However, parsing the AST must be a user's task since the language server is supposed to know nothing about the syntax of the given language.

Some languages may represent programs as a list of declarations. From the scope-tracking view, these are programs where the scopes gets injected manually to an expression according to the whole project's context. The program, hence, is defined as a set of expressions, not a nested structure. Scope management in these scenarios becomes more sophisticated compared to the nested one-expression case. Yet it's still possible to produce a Free Foil AST leaving a single expression representing a whole program afterwards, turning it into a list of expressions each with a distinct scope is also a solution. In terms of an agnostic language server producing a list of trees from a given String input is a cheap yet powerful modification comparing to limiting it to a single element allowing for more user customizations without a requirement to extend the second-order abstract syntax signature.

The function producing Free Foil ASTs from a given input is defined as the following:

```
buildAsts :: String -> [SomeScopeWithAST binder sig]
```

SomeScopeWithAST is parametrized by a binder and sig but it also introduces an existential variable n which makes any function referring to the SomeScopedAST independent of the concrete phantom type parameter n while also preserving the relation of the AST binder sig n to the scope Scope n.

```
data SomeScopeWithAST binder sig where
  SomeScopeWithAST :: (Distinct n)
    => Scope n
    -> AST binder sig n
    -> SomeScopeWithAST binder sig
```

3.2.2 Agnostic Tree Traversal

A central challenge in building a language-agnostic server on top of Free-Foil is traversing an AST binder sig n without knowing the concrete types of binder or sig. A term is either a free variable Var or a Node whose immediate children are encoded by the signature:

```
data AST binder sig n where
  Var :: Foil.Name n -> AST binder sig n
  Node
    :: sig (ScopedAST binder sig n) (AST binder sig n)
    -> AST binder sig n
data ScopedAST binder sig n where
  ScopedAST :: binder n l -> AST binder sig l -> ScopedAST binder sig n
```

The first type argument of sig carries scoped children, sub-expressions placed under a binder of the form ScopedAST binder sig n, while the second carries plain sub-expressions AST binder sig n. Without additional constraints, the signature is entirely opaque to the server.

The Bifoldable type class provides the ability to fold over both argument positions of sig in a single pass:

```
class Bifoldable t where
  bifoldl :: (c -> a -> c) -> (c -> b -> c) -> c -> t a b -> c
```

A call to bifoldl dispatches two callbacks, one for scoped children and one for plain sub-expressions, threading an accumulator through all of them. Applying this recursively yields a fully generic depth-first traversal requiring no knowledge of the grammar:

```
traverseAST
  :: (Bifoldable sig)
  => (forall n. Foil.Name n -> acc -> acc)
  -> (forall n. AST binder sig n -> acc -> acc)
  -> acc -> AST binder sig n -> acc
traverseAST onVar _ acc (Var name) = onVar name acc
traverseAST onVar onNode acc (Node sig) =
  bifoldl
    (\a (ScopedAST _binder body) -> traverseAST onVar onNode a body)
    (\a subterm -> traverseAST onVar onNode a subterm)
    (onNode (Node sig) acc)
    sig
```

The constraint `Bifoldable sig` is the only structural requirement for a complete read-only walk of the tree.

Complementarily, the `Bifunctor` type class provides `bimap`, which transforms both child positions of `sig` simultaneously:

```
class Bifunctor t where
  bimap :: (a -> c) -> (b -> d) -> t a b -> t c d
```

This enables a generic rewriting pass over the AST, where every node is rebuilt after its children have been transformed:

```
mapAST
  :: (Bifunctor sig)
  => (forall n. AST binder sig n -> AST binder sig n)
  -> AST binder sig n -> AST binder sig n
```

```
mapAST _ (Var name) = Var name
mapAST f (Node sig) = f $ Node $
  bimap
    (\(ScopedAST binder body) -> ScopedAST binder (mapAST f body))
    (mapAST f)
    sig
```

When a particular algorithm additionally needs to extract information from a node, such as a source location or a construct's display name, the signature can be required to implement a further type class that exposes that information.

Together, folding with Bifoldable, mapping with Bifunctor, and targeted extraction via ad-hoc type classes cover the full space of operations a language server needs to perform on the AST. Every server-side algorithm can be formulated as a combination of these, with the constraints serving as a precise, type-checked contract between the generic server and the concrete language implementation. The server core requires no modification when a new language is plugged in; the user only provides type class instances alongside their concrete binder and sig.

3.2.3 Locating a Node

Mapping a cursor position, a line and column pair, to the narrowest AST node that covers it is the starting point for nearly every language server feature. Go-to-definition, hover messages, and rename all begin by answering the question of what the user is pointing at. The server cannot hard-code how location information is stored, because it knows nothing about the concrete grammar, so two type classes expose it from the two structural roles a user type can play.

Making signature nodes locatable. The RangedSig class asks a fully-applied signature value to report its source range:

```
class RangedSig sig where
  range :: sig (ScopedAST binder sig n) (AST binder sig n) -> Maybe Range
  range = const Nothing
```

The default returns Nothing, so a user who has not yet annotated any constructor still gets a server that compiles and runs; the position search simply will not descend past un-ranged nodes.

Making pattern nodes locatable. Binder patterns have kind S -> S -> *, so Data.Foldable cannot be derived for them. A dedicated class is therefore needed:

```
class FoldablePat pat where
  foldrPat :: (SomePattern pat sig -> r -> r) -> r -> pat n l -> r
  rangePat :: pat n l -> Maybe Range
  binderOf :: pat n l -> Maybe (NameBinder n l)
```

foldrPat walks the sub-patterns of a composite binder; rangePat returns its source range; and binderOf retrieves the single NameBinder at this level of the pattern, if any. All three methods have placeholder defaults that produce empty results, keeping the class optional to launch the language server but required to support most of the LSP features.

Pattern structure requirement. The return type of binderOf is Maybe (NameBinder n l), reflecting how Free Foil represents scope transitions. A binder pat n l moves from scope n to scope l through a single NameBinder n l, introducing

exactly one name at that level. Multiple bound names are accumulated by chaining binder levels: the output scope of one level becomes the input scope of the next. A pattern that binds several names in parallel must therefore be structured as a linked sequence of single-binder levels rather than a flat node holding several names simultaneously; only then can foldrPat visit each introduced name independently and binderOf return a well-typed NameBinder at each step.

The Var wrapper problem. A Free-Foil Var carries only a Name, an integer, and no annotation of any kind. Source ranges can be attached exclusively to Nodes through the signature. A plain Var is therefore invisible to any position-based search: the server cannot locate it, and neither can it retrieve its position after the fact.

The natural remedy is to introduce a dedicated var-wrapper constructor in the signature: a Node whose sole child is the variable it wraps. Because the wrapper is a Node it can carry a range; because it has exactly one Var child, the server can reliably extract the underlying Name from it. The user is responsible for inserting these wrappers when converting concrete syntax to the Free-Foil AST; a language implementation that omits them will find name extraction unreliable, making go-to-definition, hover look-up for variables, and rename non-functional, while features that do not depend on name resolution, such as diagnostics and highlighting, continue to work.

Range testing and the search algorithm. With location information in place, testing whether a position falls inside a range reduces to boundary comparisons on lines and columns:

```
inRange :: Position -> Range -> Bool
```

```
inRange (Position l c) (Range (Position x y) (Position x' y')) =
  let startsOK = ((l /= x) || (c >= y))
      endsOK = ((l /= x') || (c <= y'))
  in l >= x && l <= x' && startsOK && endsOK
```

The predicate handles positions that share a line with a range boundary by additionally checking the column.

findNarrowest combines RangedSig, FoldablePat, and Bifoldable to walk the AST and return the smallest node or pattern covering the requested position:

```
findNarrowest ::
  ( Bifoldable sig
  , RangedSig sig
  , CoSinkable binder
  , FoldablePat binder
  , Distinct n )
  => Position
  -> AST binder sig n
  -> Maybe (Either
      (SomePattern binder sig, SomeAST binder sig)
      (SomeAST binder sig))
```

The result is Nothing when no ranged node covers the position. Otherwise a Left value indicates the narrowest match is a pattern (together with its enclosing AST node for context), while Right means it is a plain signature node. The algorithm uses Bifoldable to fold over the scoped and plain children of each Node, recursing only into children that pass the inRange test, and keeping whichever candidate has the smallest covering range.

Once the narrowest node is found, extractName attempts to read a Name from it by looking for the single Var child a var-wrapper node is expected to contain:

```
extractName :: (Bifoldable sig, Distinct n, CoSinkable binder)
  => AST binder sig n -> Maybe (SomeName binder sig)
```

A naked Var passed directly returns Nothing, because the server expects to receive a Node, but a var-wrapper node yields Just the wrapped Name. The pair of findNarrowest and extractName is the shared prologue for all name-sensitive operations: locate the narrowest node, then ask whether it wraps a variable.

3.2.4 Name Definition and References

The two most prominent name-sensitive LSP features, go-to-definition and rename, share a common core: given a Name extracted from the node under the cursor, find the binder that introduces it, and then collect every node in the binding's scope that refers back to it. Both tasks require a traversal that is aware of how scope evolves as it descends the tree.

Locating a definition. A Name in Free-Foil is an integer indexed by a phantom scope parameter n. Membership is cheap: n `member` scope returns True exactly when n belongs to scope. The definition search exploits this by starting at the tree's root with its initial scope and descending into ScopedAST children, extending the running scope with each binder it crosses. The moment the extended scope contains the target name, the most recently crossed binder is the one that introduced it, and its associated body is the subtree in which that name is live.

The search is split across two functions. findDefinition is the entry point. It first checks whether the name is already a member of the starting scope and returns Nothing if so, because a name that is already free in the current expression has no definition inside this tree:

```
findDefinition isValid (SomeName n) scope ast
  | n `member` scope = Nothing
  | otherwise =
      findDefinition' isValid (SomeName n) scope ast
```

findDefinition' is the recursive worker. Using Bifoldable, it folds over each ScopedAST binder body child of a Node: the binder is used to extend the running scope, membership is checked, and if the name now belongs to the extended scope the binder and body are returned as the result. When the name is still absent from the extended scope, the worker recurses into the body. Plain (non-scoped) children are recursed into without changing the scope at all.

Both functions accept an additional validity predicate:

```
findDefinition ::
  ( Distinct n
  , Bifoldable sig
  , CoSinkable binder )
  => (forall m .
        sig (ScopedAST binder sig m)
            (AST binder sig m)
        -> Bool)
  -> SomeName binder sig
  -> Scope n
```

```
  -> AST binder sig n
  -> Maybe
       ( SomePattern binder sig
       , SomeAST binder sig )
```

The predicate tests a fully-applied signature node to decide whether a candidate binder should be accepted. Its primary purpose is to disambiguate parallel scopes. Two sibling ScopedAST branches can independently introduce names with the same integer identity (since each starts from the same parent scope). Without additional filtering, the traversal might return the wrong binder. A natural disambiguator is range coverage: a binder qualifies only if its enclosing node contains the cursor position, which at most one sibling can satisfy if source ranges are non-overlapping. The predicate is caller-supplied and language-agnostic; the functions themselves impose no constraint on what criterion is used.

Collecting references. After the definition is found, the rename and find-all-references operations need every node inside the binding body that refers to the same name. Two functions handle this, mirroring the definition-search pair.

findRefs' tests the current node and then recurses into its children. A node counts as a reference if it is a var-wrapper whose single Var child carries the same integer identity as the target name:

```
findRefs' ::
  ( Distinct m
  , Bifoldable sig
  , CoSinkable binder )
  => SomeName binder sig
```

-> AST binder sig m
-> [SomeAST binder sig]

The integer comparison is sound because the binder that introduces the target name is an ancestor of every node in its body, so the name is already in scope throughout the subtree. Foil's withFresh always allocates an identifier beyond those currently in scope, so it can never reassign the target's identifier to another binder inside the body; and the parallel-sibling collisions discussed in Section 4 only reuse identifiers allocated within the body, which are necessarily distinct from the already-in-scope target. No unrelated occurrence can therefore share the target's identifier.

findRefs is the symmetric counterpart: it skips testing the node it receives and dispatches directly to findRefs' on all children via bifoldMap. The split avoids double-counting when a caller already holds a root node known to be a reference and is only interested in collecting its descendants.

Composing definition and reference search. The two pairs are designed to compose. A go-to-definition query calls findDefinition to obtain the binder pattern, then extracts its source range via FoldablePat. A rename query calls findDefinition to obtain the definition's body, passes that body to findRefs, and collects the range of every returned wrapper-node to build the complete set of edit locations. The same combination is equally applicable to a find-all-references request. Since neither function depends on any language-specific type class beyond Bifoldable and CoSinkable, both pairs are available for any Free-Foil AST without modification to the core server. The concrete handler logic that wires these building blocks into LSP responses is discussed in the implementation chapter.

3.2.5 Type Checking

Type checking in the agnostic language server follows a bidirectional strategy: type expectations are propagated downward from a parent node to its children, while inferred types are returned upward as results. The user supplies the language-specific deduction rules; the server handles the recursive descent and name-map bookkeeping. Three type classes divide this responsibility.

Reading types back. Once a node or pattern has been annotated with a type, two simple classes expose that annotation:

```
class TypedSig sig ty where
  ty :: sig (ScopedAST binder sig n) (AST binder sig n) -> ty
class TypedPat pat ty where
  patTy :: pat n l -> ty
  addPattern :: pat n l -> NameMap n ty -> NameMap l ty
```

TypedSig extracts the inferred type from any fully-applied signature node. TypedPat does the same for binder patterns; its second method, addPattern, extends the running name map with the types of the names the pattern introduces, making them available for type-checking the body.

Specifying deduction rules. The central class is TypeDeductiveSig:

```
class TypeDeductiveSig sig sig' ty binder where
  deduceType
    :: ty
    -> sig
       (ty, ty -> (ScopedAST binder sig' n, ty))
```

```
      (ty -> (AST binder sig' n, ty))
  -> sig' (ScopedAST binder sig' n) (AST binder sig' n)
```

The first argument ty is the expected type for the node as a whole, namely what the node's context demands. The second argument is the signature with its children already transformed into continuations: each plain child becomes a function ty -> (AST ..., ty), and each scoped child becomes a pair of its pattern type together with a similar function over the body. Calling a child's continuation with some type expectation produces both the recursively-checked subtree and its inferred type. The implementation of deduceType is therefore a pure decision procedure: given the expected type and the child continuations, decide which type to demand of each child, force those continuations, and assemble the output node with a freshly annotated second field.

Orchestrating the recursion. The generic typecheck function ties the classes together:

```
typecheck ::
  ( Bifunctor sig
  , TypeDeductiveSig sig sig' ty binder
  , TypedSig sig' ty
  , TypedPat binder ty
  , Distinct n
  , CoSinkable binder )
  => NameMap n ty -> ty -> AST binder sig n -> AST binder sig' n
typecheck nm typ = \case
  Var n -> Var n
```

```
Node sig -> Node $ deduceType typ $ bimap
  (\(ScopedAST binder a) -> case assertDistinct binder of
    Distinct ->
      let f = first (ScopedAST binder)
            . check (addPattern binder nm) a
      in (patTy binder, f))
  (check nm)
  sig
```

A Var node carries no type annotation and is left unchanged; its type will be looked up in the name map whenever a parent's continuation requests it. For a Node, bimap converts every child in the raw signature into a continuation: plain children become check nm, which recursively calls typecheck and then reads back the inferred type; scoped children are paired with the pattern type from patTy and wrapped so that calling them extends the name map via addPattern before the recursive descent. The transformed signature is then handed to deduceType, which contains all language-specific logic.

Example: checking a function application. Consider the expression (\x : Int . x) 42 under the expectation that the whole expression has type Int. In a concrete implementation, this is an AppF node whose two children are the lambda and the integer constant. After bimap runs, deduceType receives:

```
AppF a a'
  xOf   -- :: Type -> (AST ..., Type)
  yOf   -- :: Type -> (AST ..., Type)
```

The AppF clause then proceeds as follows:

```
AppF a a' xOf yOf ->
  let (y, yTy) = yOf anyTypeEmp
      (x, xTy) = xOf (FunType funTypeAnnEmp yTy exTy)
      xReturnTy = case xTy of
        FunType _ _ r -> r
        _ -> anyTypeEmp
      (t, errs) = resolveType exTy xReturnTy
  in AppF a (NodeAnn a' t errs) x y
```

First, the argument 42 is checked with no expectation (anyTypeEmp), yielding type Int. Then the function \x : Int . x is checked under the expectation Int -> Int, constructed from the argument type just inferred and the overall expected type Int. The lambda confirms type Int -> Int, so its return type is Int. The last step is to compare the inferred return type against the overall expectation. This requires a user-supplied resolveType function that takes an expected type and an inferred type and returns the resolved type together with any mismatch errors. A straightforward implementation simply checks equality:

```
resolveType :: Type -> Type -> (Type, [String])
resolveType expected inferred
  | expected == anyTypeEmp = (inferred, [])
  | expected == inferred   = (inferred, [])
  | otherwise = (expected,
      ["expected " ++ show expected
       ++ ", got " ++ show inferred])
```

When the expected type is absent (anyTypeEmp), the inferred type is accepted outright. When they match, no error is recorded. Otherwise, a mismatch

message is stored in the annotation's error list, which later surfaces as a diagnostic. More sophisticated languages may unify or subtype rather than check equality, but the interface is the same: given two types, produce a resolved type and any errors.

3.2.6 Code Highlighting

Code highlighting is based on semantic tokens: a flat list of tokens, each annotated with a type (keyword, variable, function, number, etc.) and a source range, which the editor colors according to its theme. Unlike syntax-based highlighting, which operates on raw text, semantic tokens are produced from the fully parsed AST, enabling token types that depend on context, such as distinguishing a local variable from a function name.

Why two type classes are needed. An AST has two structurally distinct components that may carry token information: the signature nodes in sig and the binder patterns in binder. A lambda's keyword token belongs to the node, while the pattern variable's token belongs to the binder. Because both are opaque to the server, two separate type classes expose them:

```
class TokenizableSig sig where
  tokenize
    :: sig ScopedASTTokens ASTTokens
    -> [SemanticTokenAbsolute]
  tokenize = const []
class TokenizablePat pat where
  tokenizePat :: pat n l -> [SemanticTokenAbsolute]
  tokenizePat = const []
```

Both methods have placeholder defaults that return an empty list, so a user who has not yet implemented highlighting still gets a server that compiles and runs correctly, and the feature degrades gracefully to no highlighting.

Token list types. The child positions of sig are replaced by their already-computed token lists before tokenize is called. This requires naming those replacement types:

```
type ASTTokens = [SemanticTokenAbsolute]
type ScopedASTTokens = (ASTTokens, ASTTokens)
```

ASTTokens stands for the token list produced by a plain child. ScopedASTTokens is a pair: the first component carries the tokens emitted by the binder pattern (tokenizePat), and the second carries those emitted by the body (tokenizeAST). Together they give tokenize access to every token from every descendant, in addition to whatever tokens the node itself contributes (e.g. keyword punctuation stored in the node's annotation).

Generic traversal. The recursive descent is implemented once by the library using Bifunctor:

```
tokenizeAST
  :: (Bifunctor sig, TokenizableSig sig, TokenizablePat binder)
  => AST binder sig n
  -> [SemanticTokenAbsolute]
tokenizeAST = \case
  Var{} -> []
  Node node ->
```

```
  tokenize $ bimap tokenizeScopedAST tokenizeAST node
where
  tokenizeScopedAST (ScopedAST pat body) =
    (tokenizePat pat, tokenizeAST body)
```

bimap replaces every child in the signature simultaneously, substituting scoped children with their ScopedASTTokens pairs and plain children with their ASTTokens lists, before the result is handed to the user-supplied tokenize. A bare Var contributes no tokens on its own: location and token type are attached to the enclosing var-wrapper node through the signature, which is handled by the Node branch.

Ordering constraint. Semantic tokens are encoded as position deltas, where each token is positioned relative to the previous one rather than in absolute coordinates. A consequence is that any token whose start position precedes the end of the token before it in the list is silently dropped. The token list produced by tokenizeAST must therefore be in source order.

The bimap-based design delegates ordering responsibility to the user's tokenize implementation. Given the pre-computed token lists for each child, the user concatenates them in the order they appear in the source. This is natural: the user knows the grammar and can write the children's lists in left-to-right order directly. An alternative, collecting all tokens and sorting by position, would also be correct but incurs an $O(K \log K)$ cost and obscures the structure. The bimap approach keeps the algorithm $O(K)$ and lets the user exploit grammatical knowledge to avoid the sort entirely.

3.2.7 Diagnostics

Diagnostics are the inline error, warning, and hint messages that an editor displays alongside source code, such as underlines, gutter icons, and problem-panel entries. Unlike hover messages, which are produced on demand in response to a cursor position, diagnostics are pushed by the server proactively whenever the contents of a file change. The server walks the entire AST, collects a flat list of Diagnostic values, and publishes them to the editor in one notification.

Why two type classes are needed. The structural reason is identical to that for hover and highlighting. An AST contains two kinds of opaque components, signature nodes in sig and binder patterns in binder, and the server cannot inspect either without a type class. A pattern such as a let-binding might carry a type-annotation mismatch diagnostic, while a node such as a function application might carry a return-type error. Keeping the two roles separate lets the user attach diagnostics to whichever component holds the relevant information:

```
class DiagnosableSig sig where
  diagnose
    :: sig (ScopedAST binder sig n) (AST binder sig n)
    -> [Diagnostic]
  diagnose = const []
class DiagnosablePat pat where
  diagnosePat :: pat n l -> [Diagnostic]
  diagnosePat = const []
```

Both methods default to returning an empty list. A user who has not yet implemented any diagnostic logic still obtains a server that compiles and runs; the

feature degrades gracefully to silence.

What the user provides. diagnose and diagnosePat return fully assembled Diagnostic values. A Diagnostic is a structured LSP record carrying, at minimum, a source range and a human-readable message, and optionally a severity level (error, warning, information, or hint), a diagnostic code, and a source label. Because the appropriate severity and range differ by construct and by language, the user is responsible for building the record; the server provides no further wrapping. The type-checking pass described in Section 3.2.5 is a natural source of diagnostics: the user-supplied type-resolution function stores any mismatch messages directly in node annotations, from which diagnose can extract them.

Generic traversal. The collection pass mirrors the structure used for highlighting. diagnoseAST uses Bifoldable to visit every node and every binder pattern in a single depth-first walk, accumulating the lists produced at each step:

```
diagnoseAST ::
  ( CoSinkable binder
  , Bifoldable sig
  , DiagnosableSig sig
  , DiagnosablePat binder )
  => AST binder sig n -> [Diagnostic]
diagnoseAST = \case
  Var _ -> []
  Node n -> diagnose n ++ bifoldMap
    (\(ScopedAST binder a) ->
      diagnosePat binder ++ diagnoseAST a)
```

diagnoseAST

n

A bare Var contributes nothing; its diagnostic, if any, belongs to the enclosing var-wrapper node in the signature. For a Node, the traversal first calls diagnose on the node itself to collect any diagnostics the user attached at this level, then uses bifoldMap to recurse into the scoped children, calling diagnosePat on each binder and diagnoseAST on each body, and into the plain children. The result is a flat list of all diagnostics in the tree.

Running diagnoseAST over every tree stored in the file cache and concatenating the results produces the complete diagnostic list for a file, which the server then publishes in a single textDocument/publishDiagnostics notification.

3.2.8 Hover Messages

Hover messages are the documentation popups an editor displays when the user rests the cursor over a symbol. They typically show type information, a brief description, or any other language-specific annotation the implementation chooses to expose. Unlike diagnostics, which the server pushes proactively on every file change, hover content is produced on demand: the editor sends a single request carrying a cursor position, and the server must respond with a MarkupContent value, or nothing if the position does not correspond to a hoverable construct.

The production of a hover message therefore decomposes into two independent steps. First, the server identifies which AST node the cursor is pointing at by calling findNarrowest, whose algorithm was described in §3.2.2 and builds on the location infrastructure introduced there. Second, it asks that node what, if anything, it has to say. The second step is language-specific and is therefore

delegated to two type classes, one for each structural role an opaque component can play.

Why two type classes are needed. The structural reason mirrors that for diagnostics and highlighting. Hover-relevant information may reside in a signature node, for instance a function-application node annotated with the inferred return type, or in a binder pattern, for instance a let-binding pattern that stores the declared type of the bound name. Because the server cannot inspect either component directly, two separate classes expose the relevant content:

```
class HoverableSig sig where
  hoverData
    :: sig (ScopedAST binder sig n) (AST binder sig n)
    -> [String]
  hoverData = const []
class HoverablePat pat where
  hoverDataPat :: pat n l -> [String]
  hoverDataPat = const []
```

Both methods default to the empty list, so a user who has not yet implemented hover support still obtains a server that compiles and runs; the feature degrades gracefully to silence.

What the user provides. hoverData and hoverDataPat return a list of strings, namely the lines to display for that construct. The user need not deal with the LSP wire format: the library concatenates these lines into a single Markdown message and wraps it as the MarkupContent value the protocol expects, so the user supplies only the content. The type-checking pass described in Section 3.2.5 is a natural

source of that content: the annotation stored on a node after type-checking already contains the inferred type, which hoverData can return as a line directly.

Producing the message. Once findNarrowest has identified the construct under the cursor, the server asks it for content directly: if the narrowest match is a binder pattern it calls hoverDataPat, and if it is a signature node it calls hoverData. A var-wrapper node needs no special treatment, because type checking annotates each node in place and the var-wrapper already carries the information to surface (typically the variable's inferred type), so hoverData returns it without any separate definition lookup. The lines returned by whichever method fired are concatenated into a single Markdown message and sent to the editor; an empty result yields no hover, which the editor interprets as "nothing to show here." The server core requires no modification when a new language is plugged in; the user only provides HoverableSig and HoverablePat instances alongside their concrete binder and sig.

Chapter 4

Implementation

4.1 LSP Interface

The Haskell LSP library [1] allows to easily run a language server and define its behavior using a list of handlers. A handler is a function acting in response to a request body. The library defines a finite amount of events which can be triggered by a notification or manual request, which, in terms of handlers are all, eventually, just events triggering a consequent response function. The project uses the following handler-functions:

- Initialized is triggered on language server initialization; it builds the initial AST cache for all files in the workspace and publishes diagnostics.
- TextDocumentDidOpen is triggered when a file is opened in the editor; it publishes diagnostics for the newly opened file.
- TextDocumentDidChange is triggered on every in-editor edit; it rebuilds the AST for the changed file using a debounced timer and publishes updated

[1] https://github.com/haskell/lsp

diagnostics.

- WorkspaceDidChangeWatchedFiles is triggered when watched files on disk change; it rebuilds the full workspace cache and republishes diagnostics for every file.
- TextDocumentDefinition is triggered when the user requests navigation to the definition of a symbol under the cursor.
- TextDocumentRename is triggered when the user initiates a rename operation on a symbol.
- TextDocumentSemanticTokensFull is triggered when the editor requests semantic highlighting tokens for an entire file.
- TextDocumentHover is triggered when the user hovers over a symbol and requests contextual information such as an inferred type.

4.1.1 Language Server Configuration

The library's public entry point is runLanguageServer, which accepts a single value of type LSConfiguration:

```
data LSConfiguration binder sig = LSConfiguration
  { fileExtension :: String
  , buildAsts :: String -> [SomeScopeWithAST binder sig]
  }
```

These two fields are the only strictly mandatory inputs. fileExtension names the file extension whose files the server should watch (e.g. "flan"), and buildAsts

converts raw file text into a list of scope-rooted ASTs, one per top-level declaration or independent scope root.

The remainder of the feature set is controlled entirely by type class instances on binder and sig. Most classes carry a default no-op implementation, so an integrator can enable features incrementally by providing non-trivial instances. The type-checking classes, TypeDeductiveSig, TypedSig, and TypedPat, are an exception: because they are parameterized over an unknown type ty that the library cannot infer or default, they require a complete user-supplied implementation. Table 4.1 summarizes which instances unlock which editor features.

Instance(s)	Editor feature
RangedSig, FoldablePat	Go-to-definition, rename
TypeDeductiveSig, TypedSig, TypedPat	Type checking
HoverableSig, HoverablePat	Hover messages
DiagnosableSig, DiagnosablePat	Diagnostic messages
TokenizableSig, TokenizablePat	Semantic token highlighting

TABLE 4.1
Type class instances and the LSP features they enable.

The Bifunctor and Bifoldable instances are structural requirements inherited from Free Foil and are not specific to any language server feature. F.CoSinkable is likewise a Free Foil constraint on the binder type required for scope extension during name traversal.

4.1.2 Building a Tree

The LSP.Server.Core module provides a function getRootPath fetching the root path of the current workspace. Using globDir from the System.FilePath.Glob module it is then possible to collect all files whose extension matches fileExtension from the LSConfiguration. The buildAsts field, also supplied by the user via

LSConfiguration, converts each file's text content into a list of SomeScopeWithAST values, one per top-level declaration or independent scope root. The resulting pairs of file path and program store are inserted into the cache.

4.2 Caching

The ASTs produced on initialization are cached for later usage. Every watched files change triggers ASTs rebuild.

The cache is defined as a separate module independent of Free Foil which is deliberately done to foster modular design and future extensibility to adapt caching to more complex scenarios. Caching is implemented using Software Transactional Memory (STM) [2]. The program's environment stores a mutable variable LangStore ast parametrized by the final AST type, SomeScopedAST binder sig in the current implementation. LangStore stores a Map connecting FilePaths to LangProgramStores, each, essentially, storing the list of ASTs:

```
newtype LangProgramStore ast = LangProgramStore
  { langAst :: [ast]
  }
newtype LangStore ast
  = LangStore (Map FilePath (LangProgramStore ast))
data LangEnv ast = LangEnv
  { store :: TVar (LangStore ast)
  , debounceTimers :: TVar (Map Uri ThreadId)
  , lspEnvRef
      :: IORef (Maybe (LanguageContextEnv ()))
```

[2] https://hackage.haskell.org/package/stm

```
}
```

The store field holds the cached ASTs protected by an STM transactional variable. The debounceTimers field maps each open document URI to the ThreadId of a pending rebuild timer, allowing the server to cancel a superseded rebuild when the user types again before the delay expires. The lspEnvRef field stores the LSP server environment as an IORef so that the debounce thread, which runs outside the LSP monad, can access it to push diagnostics after the rebuild completes.

Why IORef (Maybe ...) **is necessary.** The runServer callback accepts a ServerDefinition whose staticHandlers field is evaluated before the LSP handshake takes place. The LanguageContextEnv, the value needed to run runLspT and send notifications, is only produced inside the doInitialize callback, which fires after the client's initialize request arrives. The debounce thread, however, is spawned from a handler that was registered earlier, so there is a window during which the thread could in principle execute before doInitialize has stored the environment. Using a plain IORef containing the fully-initialized value would require the environment to exist at construction time, creating a chicken-and-egg dependency. IORef (Maybe (LanguageContextEnv ())) resolves this: the reference is initialized to Nothing, then atomically overwritten with Just env inside doInitialize. The debounce thread reads the reference and exits silently when it finds Nothing, a case that is only possible during the brief initialization window, and proceeds normally once Just env is available.

The handleDidChange handler therefore does not rebuild the cache immediately. Instead, it calls debounceRediagnose: any existing timer for the document is cancelled, and a new thread is spawned that waits 500 milliseconds before rebuild-

ing the AST, publishing diagnostics, and requesting a semantic-token refresh. This prevents redundant rebuilds on every keystroke while keeping the IDE's feedback responsive.

The caching module also defines the LSP monad alias used throughout the server:

```
type LSP ast = LspT () (ReaderT (LangEnv ast) IO)
```

4.2.1 Name Collisions

The definition-search algorithm described in Section 3.2.4 uses extendScopePattern to grow the running scope as it crosses each binder, and checks name membership to detect when a binder introduces the target name. A subtle issue arises when two or more ScopedAST children of the same Node are siblings, that is, when the signature contains parallel scoped branches. Because each branch starts from the same parent scope and Foil's withFresh picks the next integer past the current maximum, both branches independently extend the scope by the same integer:

```
myFunction = -- Scope n  (e.g. {})
  ( \x -> doSmth1 x  -- Scope l1 = {0}
  , \y -> doSmth2 y  -- Scope l2 = {0}
  , \x -> doSmth3 x  -- Scope l3 = {0}
  )
```

All three binders produce integer identity 0; the scopes l1, l2, and l3 are equal as integer sets. Without further information, findDefinition' using bifoldr would return the first sibling that passes the membership check, which may not

be the one containing the cursor. Comparing the raw source names stored in the AST would still confuse the first and third branches (both named x), so that is not a solution either.

The chosen approach is to thread a validity predicate through findDefinition'. When the definition search encounters a candidate ScopedAST binder body, it applies the predicate to the node that encloses it before accepting the result. For go-to-definition and rename, the predicate is nodeCovers pos: a candidate is valid only if its surrounding signature node's source range contains the cursor position pos. Since findNarrowest has already established that the cursor falls inside the var-wrapper being queried, and since non-overlapping source ranges ensure that at most one of the parallel branches can contain the cursor, the predicate admits exactly the correct branch. No extra traversal or subtree search is required; the same single pass that searches for the binder simultaneously discards the wrong siblings.

An alternative that was considered but not adopted is to make name generation aware of parallel branches during AST construction. A customized scope type OffsetScope could carry an integer offset representing how many names have already been allocated by preceding sibling branches, producing globally distinct integers:

```
toAst :: Distinct n
  => OffsetScope n
  -> Map String (Name n)
  -> RawTerm
  -> (MyAst n, Int)
toAst scope env = \case
  RawPair l r ->
```

```
let (astL, lShift) = toAst scope env l
    (astR, rShift) =
      toAst (shiftScope scope lShift) env r
in (Node (Pair astL astR), rShift)
```

This approach eliminates collisions at the source but introduces a coupling between sibling branches: each branch's scope depends on the names allocated by all preceding siblings, which prevents independent (or parallel) construction of the child nodes. It also requires replacing Foil's standard `withFresh` with a custom variant throughout the user's AST builder. The position-based predicate approach requires no modifications to either the library or the user's AST construction code, keeping the integration surface small.

4.3 Example Project

The agnostic language server was tested by integrating it with two toy programming languages: LambdaPi, a dependently typed lambda calculus with Π-types, and Flan (Functional LANguage). Flan exercises the full feature set of the library: type checking, diagnostics, hover messages, semantic highlighting, go-to-definition, and rename; it is the primary integration example. LambdaPi serves as a minimal counterpart: it provides only `TokenizableSig` and `TokenizablePat` instances for syntax highlighting, with all other classes left at their no-op defaults, showing how little is needed to reach a first running server.

4.3.1 Signature and Annotations

Flan is a strictly typed functional programming language with lambda abstraction, let-bindings, pairs, conditionals, and integer, string, and boolean literals. Its signature has two annotation type parameters: a for source positions and a' for node-level derived data:

```
data FlanF a a' s t
  = VarF a a' t String
  | AppF a a' t t
  | LamF (LamAnn a) a' s
  | LetF (LetAnn a) a' s s
  | PairF (PairAnn a) a' t t
  | IfF (IfAnn a) a' t t t
  | ConstF a a' FlanConst
  | ErrorF a a' String
```

The a parameter (a BNFC source position) is present on every constructor; for nodes that carry multiple keywords, such as LamF, which has a lambda token and a return-type annotation token, a dedicated record (LamAnn, LetAnn, etc.) groups the per-keyword positions. The a' parameter is the node annotation used for derived information.

AST construction proceeds in two phases. In the first phase both parameters are instantiated with the raw source position type PosAnn, yielding FlanF PosAnn PosAnn. In the second phase the type-checker replaces a' with NodeAnn:

```
data NodeAnn
  = NodeAnn (Maybe (Int, Int)) FlanType' [String]
```

NodeAnn stores the node's end position (used to construct a Range together with the start position in a), its inferred type, and any type-error messages accumulated during type checking. The final AST has type FlanF PosAnn NodeAnn and carries both positional and type information without requiring a separate pass.

Flan's binder is FlanPattern, a recursive algebraic type that supports wildcard, variable, and pair patterns. Free Foil's CoSinkable typeclass is derived for it, and the library's pattern-binding machinery allows binding zero, one, or several names in a single node without any changes to the core server.

```
data FlanPattern a ty (n :: S) (l :: S) where
  PatternWildcard :: a -> a -> ty
    -> FlanPattern a ty n n
  PatternVar :: a -> a -> ty -> String
    -> NameBinder n l -> FlanPattern a ty n l
  PatternPair :: PatternPairAnn a -> a -> ty
    -> FlanPattern a ty n i
    -> FlanPattern a ty i l
    -> FlanPattern a ty n l
```

Framework requirements on user-defined types. These two definitions satisfy the two structural requirements the framework imposes on user-defined AST types. VarF serves as the var-wrapper: it is a Node constructor whose single plain child is a variable occurrence, giving the server a range-bearing handle on every name use in the source. PatternPair chains its two sub-patterns through an intermediate scope i rather than storing two binders in a single flat constructor. This ensures that binderOf can return a single NameBinder at each level and that foldrPat can walk the pattern recursively, extracting one name per step. A

language implementation that represents variable occurrences as bare Var nodes, or that stores multiple binders side-by-side in a single flat pattern constructor, cannot satisfy the FoldablePat and RangedSig contracts; go-to-definition, hover look-up, and rename will be non-functional for those constructs.

4.3.2 Type Class Instances

To activate the full feature set, the Flan integration implements the following type class instances defined by the library:

- RangedSig (FlanF PosAnn NodeAnn) and FoldablePat (FlanPattern PosAnn FlanType'), required for go-to-definition and rename; they supply source ranges for nodes and binders.
- TypeDeductiveSig, TypedSig, and TypedPat together describe the bidirectional type-checking rules used to populate NodeAnn with the inferred type.
- DiagnosableSig and DiagnosablePat collect type errors and unbound-symbol errors stored in NodeAnn into LSP Diagnostic values.
- HoverableSig and HoverablePat construct hover content from the inferred type and error messages stored in each node's NodeAnn.
- TokenizableSig and TokenizablePat produce a list of SemanticTokenAbsolute values, assigning token types (variable, function, keyword, number, string) to each sub-expression and keyword token.

The entry point wires everything together via LSConfiguration:

```
runFlanLS :: IO ()
runFlanLS = runLanguageServer LSConfiguration
  { fileExtension = "flan"
  , buildAsts = buildAsts'
  }
```

where buildAsts' invokes the BNFC-generated lexer and parser, converts the concrete syntax tree to a Free-Foil AST using the position annotation, and then applies the type-checker to produce the fully-annotated NodeAnn tree.

4.3.3 Visual Studio Code Demo

A minimal Visual Studio Code extension hosts the compiled Flan language server executable and registers it for .flan files. The extension requires no language-specific logic: it forwards all LSP traffic to the server process. Through this setup the extension demonstrates go-to-definition, rename, hover with inferred types, squiggly-line diagnostics for type errors and unbound symbols, and syntax highlighting driven by semantic tokens, all enabled by the type class instances above, with no additional hand-written IDE glue.

Chapter 5

Evaluation and Discussion

This chapter assesses the agnostic language server framework against the evaluation criteria established in Section 3.1.2, analyzes the automation it delivers and the computational cost of its algorithms, and identifies current limitations together with directions for future work.

5.1 Evaluation against Design Criteria

5.1.1 Feature Completeness

The framework supports all six features from the criterion: go-to-definition, symbol renaming, hover messages, diagnostics, type checking, and syntax highlighting (Table 4.1). Each is activated by the corresponding type class instances. The Flan language server exercises all six simultaneously: go-to-definition and rename resolve variable uses to their introducing binders; hover messages display inferred types; diagnostics surface type errors and unbound symbols; semantic tokens drive syntax highlighting in VS Code.

5.1.2 Minimal Entry Threshold

Every method in the non-type-checking classes carries a default returning a neutral value, so the only strictly mandatory inputs are fileExtension and buildAsts in LSConfiguration. Features degrade gracefully: supplying only RangedSig and FoldablePat immediately enables go-to-definition and rename; each subsequent pair of instances adds the next feature without touching what is already working. The type-checking classes are a partial exception: because they are parameterized over a user-supplied type ty the library cannot default, omitting them simply means no type-checking support, and all other features are unaffected.

5.1.3 Correctness

Scope-tracking correctness rests on two mechanisms. Free Foil's phantom type parameter n :: S makes scope violations a compile-time error: a Name n cannot be compared against a Scope m for a different m, so the server inherits these guarantees for free. The parallel-scope collision problem, in which sibling ScopedAST branches independently produce names with the same integer identity, is resolved by the position-based validity predicate in findDefinition: because source ranges are non-overlapping, at most one sibling covers the cursor, and only that branch is accepted. This scope-level correctness is what Free Foil enforces by construction; resolving a name to its definition additionally relies on the structural contracts of Sections 3.2.2 and 4, namely var-wrapper nodes around variable occurrences and single-binder-per-level patterns, which the library cannot verify and which the integrator must honor for go-to-definition, rename, and variable hover to function.

5.1.4 Language Agnosticism

Two language servers were built on the library without modifying any core module: LambdaPi (a dependently typed lambda calculus with Π-types) and Flan (a functional language with let-bindings, pair patterns, and conditionals). Their signature constructors, binder types, parsers, and type systems share no code; the integration surface in each case consists only of type class instances and the two LSConfiguration fields.

5.1.5 Worst-Case Complexity

The criterion requires every server operation to run in at most linear time in the AST size ($O(K)$). This is satisfied: the analysis of Section 5.3 shows that every handler is bounded by $O(K)$, and the library introduces no super-linear step of its own.

Several operations do better than this worst case, which is a welcome bonus rather than a requirement. Position-only queries, hover and go-to-definition, descend a single root-to-target path instead of the whole tree, so for fixed-arity grammars they run in $O(L)$ in the tree depth (Section 5.3); since $L \leq K$, and substantially so for shallow, wide trees, this is a real improvement where it applies.

As subjective corroboration, timing via VS Code's LSP debug log shows individual handler responses in the range of 10 ms on sample Flan programs, which is fast enough to feel instantaneous in the editor. This is not a controlled benchmark, but together with the complexity bounds it indicates that the resulting performance is comfortably adequate for the framework's intended use; a systematic empirical characterization across controlled file sizes is left as future work.

5.1.6 VS Code Integration

The criterion is satisfied by the Flan extension described in Section 4: a thin wrapper that registers the compiled server binary for .flan files and forwards all LSP traffic to it. Go-to-definition, rename, hover tooltips, squiggly-line diagnostics, and semantic-token highlighting are all accessible through VS Code's standard interface without additional configuration.

5.2 Automation Analysis

A natural measure of the automation a framework delivers is the number of user-defined functions required to activate a given feature. The fewer functions a feature demands, the more of its implementation the library has absorbed. These functions can be classified by what they operate on:

- functions on sig, concerned with the language's signature constructors;
- functions on binder, concerned with the language's binder patterns;
- functions concerned with neither.

Because the library's automation is achieved by internalizing operations on the Free Foil AST, the number of user functions in the third category directly indicates how much tree-traversal and scope-management work has been absorbed into the library.

Table 5.1 lists the type class methods a user must provide to activate each feature, counting only new methods each feature adds in an incremental activation order.

Feature	Methods to implement	New
Minimal server (no features)	buildAsts	1
Go-to-definition	range, foldrPat, rangePat, binderOf	4
Rename	(same as go-to-definition)	0
Hover	hoverData, hoverDataPat	2
Diagnostics	diagnose, diagnosePat	2
Highlighting	tokenize, tokenizePat	2
Type checking	deduceType, ty, patTy, addPattern	4
Total		**15**

TABLE 5.1
User-defined functions required per feature, with incremental new additions.

Activating all supported features requires 15 user-defined functions. Fourteen of these are type class methods operating on sig or binder; only buildAsts involves neither. This single exception implements the language-specific parsing and AST construction step, which is irreducibly language-specific and cannot be automated away. All tree traversal, scope extension, name-membership testing, and handler wiring is handled by the library without user involvement. The fact that only one function sits outside the sig/binder category confirms a high level of automation.

This count measures the framework's integration surface, the number of distinct extension points a language author must fill, rather than implementation effort. Two of these inputs, buildAsts and deduceType, concentrate the genuinely language-specific complexity, parsing and the type system respectively, and may each be substantial; the remaining methods are typically small accessors over the signature. The metric therefore reflects how much tree-traversal and scope-management work the library absorbs, which is precisely the automation being claimed, and not the total effort of integrating a language.

Two further observations follow from Table 5.1. First, the four methods of RangedSig and FoldablePat together unlock two non-trivial LSP features, go-to-

definition and rename, simultaneously. Both features share the same traversal and name-resolution primitives, so implementing them once suffices for both. This pair is the most automation-efficient entry point for a language integrator seeking immediate IDE functionality.

Second, type checking has the highest incremental cost, four methods that cannot be defaulted, but also the feature that most directly benefits from Free Foil's scope-tracking infrastructure. The user supplies only the language-specific deduction rules via $\mathrm{deduceType}$; the library provides the recursive descent, the name map, and the continuation-passing interface.

5.3 Complexity Analysis

Let K denote the total number of nodes in the AST and L the length of a root-to-target path (at most the tree depth, so $\mathrm{L} \leq \mathrm{K}$). Each operation visits every node at most a constant number of times, so all server operations are $O(K)$ in the worst case; for the whole-tree operations, namely type checking, diagnostics, highlighting, and reference collection during rename, this is also tight, since each must inspect every node to produce its result.

Operation	Worst-case	Dominant step
AST construction	$O(K)$	User-supplied; linear parse assumed
Go-to-definition	$O(K)$	Narrowest-node search + binder search ($O(L)$ with pruning)
Rename	$O(K)$	Reference collection after the binder search
Type checking	$O(K)$	Single-pass bimap descent
Hover	$O(K)$	Narrowest-node search only ($O(L)$ with pruning)
Diagnostics	$O(K)$	Full tree walk
Highlighting	$O(K)$	Full tree walk

TABLE 5.2
Worst-case time complexity of each server operation.

Position-based queries, hover and go-to-definition, usually do better than this worst case. Since each node carries a source range and sibling ranges do not overlap, the search descends only into the child covering the cursor and prunes the rest, following one root-to-target path of length L ($\ll \mathrm{K}$ for shallow or balanced trees). The per-step work is the number of children examined; for a fixed, non-variadic signature this is a grammar constant, so each step is $O(1)$ and the query is $O(L)$. The sole exception is a variadic, list-valued constructor with $\Theta(K)$ children, where scanning them is linear and the query falls back to $O(K)$. Fixed-arity grammars such as Flan never trigger this; a list of top-level declarations is handled outside the tree as separate ASTs.

Such a variadic node could be re-encoded as a nested, cons-like chain, restoring bounded arity by construction and making the bound hold unconditionally in the encoded depth L'. This does not reduce the work: locating one child among N still costs $\Theta(N)$, and the chain has depth N, so $L' \geq L$, but it shows the $O(K)$ case is a matter of representation, not an algorithmic limit. A genuinely sub-linear scan would instead index the children by range, as noted under future work in Section 5.4.

The linear bound for type checking, diagnostics, and highlighting assumes that the user-supplied type class methods operate in $O(1)$ per node. If a user's $\mathrm{deduceType}$ or $\mathrm{diagnose}$ implementation performs additional work, for example resolving overloaded names by consulting a separate data structure, the effective per-node cost rises accordingly. The library itself does not introduce super-linear behavior.

Semantic tokens have an additional ordering requirement: the LSP wire format encodes positions as deltas, so any token whose start position precedes the end of the previous token in the list is silently dropped. The bimap-based token

collection delegates ordering to the user's $\mathrm{tokenize}$ implementation, keeping the algorithm at $O(K)$. The alternative, collecting all tokens and sorting by position, would cost $O(K \log K)$.

5.4 Limitations and Future Work

The following limitations are present in the current implementation.

Single-file scope. The library does not model a global or cross-file namespace. Go-to-definition and rename operate within the ASTs of a single file; if the target name is introduced in a different file, the query terminates without result. This is a property of the current implementation rather than of the S-indexed representation, which can encode such structure, for example a global scope or an import as a synthetic binder that extends the scope with the relevant names. The missing piece is the resolution and dependency machinery: supporting cross-file references would require a dependency graph over files and a mechanism for resolving names across file boundaries, both of which the integrator must supply by hand.

No dependency graph for cache invalidation. The cache is rebuilt for all files in the workspace whenever any watched file changes. For large workspaces this is wasteful. A file-level dependency graph would allow the server to rebuild only files affected by a given edit, and would also make cyclic imports detectable, which is useful for any language that forbids them.

Type inference limitations. The bidirectional type-checking interface propagates a single expected type downward from parent to child and reads a single inferred type upward. This covers straightforward bidirectional checking but does

not support constraint-based inference, where type constraints are collected during a first pass and unified in a second. Languages whose type inference relies on unification, such as Haskell or ML, cannot be accommodated without extending the framework. A related limitation is that binder types are currently sourced exclusively from the annotation stored in the pattern via patTy. Languages in which binder types are optional and inferred from usage, as in $\mathrm{let\ x = expr}$, require a richer inference mechanism.

Incremental type checking. The type-checking pass currently re-checks the entire AST whenever a file changes. An incremental approach that re-checks only subtrees affected by the edit would reduce diagnostic latency for large files, and is a natural complement to the debounce mechanism already in place.

Search complexity improvement. Position-based node lookup currently descends the relevant subtree along a single path, $O(L)$ for fixed-arity grammars. Indexing nodes by their source ranges in an interval map would reduce this to $O(\log K)$ for typical trees, at the cost of maintaining the map across edits. Similarly, a single combined traversal, descending toward the target node while simultaneously tracking candidate binders on the way back up, would replace the two separate $O(L)$ passes currently used by go-to-definition.

Default and complementary highlighting. Semantic-token highlighting currently requires explicit $\mathrm{TokenizableSig}$ and $\mathrm{TokenizablePat}$ implementations. Two improvements could lower this barrier.

First, the library could provide a default semantic highlighting pass: given $\mathrm{RangedSig}$ and $\mathrm{FoldablePat}$ instances, which many users already provide for go-to-definition and rename, it could assign a generic token type to all variable occur-

rences and binder patterns without further user input, yielding a useful baseline for free.

Second, semantic tokens and TextMate grammar highlighting are complementary. A TextMate grammar is a set of regular expressions that the editor applies directly to the raw source text, without invoking the language server at all. Because it operates on text rather than a parsed AST, it is available immediately on file open, before the server has finished building the AST, and it works even when the file contains syntax errors that prevent parsing. Pairing a lightweight TextMate grammar (covering keywords, comments, and literals) with the server's semantic tokens (covering variable and binder roles) would therefore give richer highlighting than either approach alone while keeping the server's responsibility limited to what only a parsed AST can provide.

Pattern structure constraint. The framework requires that binder patterns follow a chained structure: a pattern binding n names must be represented as n levels of $\mathrm{FoldablePat}$ nodes, each contributing exactly one $\mathrm{NameBinder}$. Patterns that store multiple names in a flat constructor are incompatible with $\mathrm{FoldablePat}$, and go-to-definition, rename, and hover for those pattern-bound names will be non-functional. This structural requirement is a consequence of Free Foil's type-level scope indexing and cannot be relaxed without changing the underlying representation.

Chapter 6

Conclusion

This thesis set out to carry Free Foil's compile-time scope safety across the LSP boundary into a single, configuration-driven language server, and the result demonstrates that this is achievable. By parameterizing the server over an opaque binder and bifunctor sig and expressing every operation in terms of Bifoldable, Bifunctor, and a small set of targeted type classes, the library absorbs all tree traversal, scope extension, and name resolution. A language integrator supplies 15 functions to activate the full feature set, only one of which, the irreducibly language-specific parser invoked by buildAsts, falls outside the sig/binder contract. Two unrelated languages were served without modifying any core module, confirming that the binder/sig parameterization achieves genuine language agnosticism. Flan exercises the full feature set and shows what a complete integration looks like; LambdaPi provides only syntax highlighting, relying on no-op defaults for everything else, and shows how little is required to reach a first running state.

The approach inherits Free Foil's central strength: scope violations become compile-time type errors rather than latent runtime bugs. Using Haskell's type system to encode the in-scope set as a phantom kind, and using the host language to

describe a target language's syntax through a second-order signature, turns a normally bespoke and error-prone layer into reusable, statically checked infrastructure. The automation and complexity analyses show that this comes at no asymptotic cost: every handler stays within $O(K)$ of the AST size, and go-to-definition and hover, which follow a single root-to-target path, drop to $O(L)$ in the tree depth for fixed-arity grammars.

The framework also has clear boundaries, though they are better understood as what it provides out of the box than as hard limits of the underlying representation. The current server operates within a single file, models lexical binding, and performs bidirectional type checking rather than constraint-based unification. None of these are forced by Free Foil itself: because S is only a phantom tag on scopes and the binder is an ordinary type parameter, non-lexical structure can be encoded indirectly, for example as a global namespace as an outermost synthetic binder or as an import that extends the scope with the imported names. What Free Foil does not supply, unlike scope graphs, is the resolution and dependency machinery itself: visibility, shadowing, qualified lookup, and cross-module name identity must be built by hand, and such encodings recover expressiveness at the cost of some of the by-construction guarantees that motivate the intrinsic approach. The boundaries are therefore the work Free Foil leaves to the integrator rather than expressive impossibilities, and they delimit the languages the current implementation serves without additional machinery.

Other frameworks that treat name resolution as a first-class, language-independent concern make different trade-offs against the same underlying goal. Scope graphs and their Statix realization in Spoofax cover non-lexical binding, imports, and rich type systems, but obtain their guarantees through meta-theoretic proof and runtime constraint solving over an extrinsic graph rather than the host

type system. Both formalize a common conception in the world of programming languages, scope, binding, resolution, and types, so that compiler and analyzer behavior can be derived once and reused, and each occupies a distinct point on the axis between expressive breadth and statically checked correctness. The two families are complementary rather than competing.

A Free-Foil-based language server is therefore already practical for study projects, proof assistants, and simple domain-specific languages, where lexical binding suffices and compiler-checked scope safety is valuable. Beyond immediate use, it serves as a reference point for how far the intrinsic, type-safe route can be carried into editor tooling, and the limitations identified here, namely cross-file resolution, dependency-driven cache invalidation, incremental type checking, and richer inference, map out concrete directions for further research. Tools of this kind, which formalize the recurring structures of programming languages and reuse them across implementations, are how the long-term cost of building language tooling is brought down; the work presented here is a step along that path, and a powerful instrument within the scope it addresses.

Bibliography cited

[1] *Language Server Protocol*. [Online]. Available: https://microsoft.github.io/language-server-protocol/.

[2] N. de Bruijn, "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser theorem," *Indagationes Mathematicae (Proceedings)*, vol. 75, no. 5, pp. 381–392, 1972, ISSN: 1385-7258. DOI: https://doi.org/10.1016/1385-7258(72)90034-0. [Online]. Available: https://www.sciencedirect.com/science/article/pii/1385725872900340.

[3] N. Kudasov, R. Shakirova, E. Shalagin, and K. Tyulebaeva, *Free foil: Generating efficient and scope-safe abstract syntax*, 2024. arXiv: 2405.16384 [cs.PL]. [Online]. Available: https://arxiv.org/abs/2405.16384.

[4] N. Kudasov, *Free Monads, Intrinsic Scoping, and Higher-Order Preunification*, To appear in *TFP 2024*, Orange, NJ, USA, 2024. arXiv: 2204.05653 [cs.LO].

[5] D. Maclaurin, A. Radul, and A. Paszke, "The foil: Capture-avoiding substitution with no sharp edges," in *Proceedings of the 34th Symposium on Implementation and Application of Functional Languages*, ser. IFL '22, Copenhagen, Denmark: Association for Computing Machinery, 2023, ISBN:

9781450398312. DOI: 10.1145/3587216.3587224. [Online]. Available: https://doi.org/10.1145/3587216.3587224.

[6] P. Neron, A. Tolmach, E. Visser, and G. Wachsmuth, "A theory of name resolution," in *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015, Proceedings*, J. Vitek, Ed., ser. Lecture Notes in Computer Science, vol. 9032, Springer, 2015, pp. 205–231. DOI: 10.1007/978-3-662-46669-8_9. [Online]. Available: https://doi.org/10.1007/978-3-662-46669-8_9.

[7] H. van Antwerpen, C. Bach Poulsen, A. Rouvoet, and E. Visser, "Scopes as types," *Proc. ACM Program. Lang.*, vol. 2, no. OOPSLA, 2018. DOI: 10.1145/3276484. [Online]. Available: https://doi.org/10.1145/3276484.

[8] L. C. L. Kats and E. Visser, "The spoofax language workbench: Rules for declarative specification of languages and ides," in *Proceedings of the 25th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2010*, Reno/Tahoe, Nevada: ACM, 2010, pp. 444–463. DOI: 10.1145/1869459.1869497. [Online]. Available: https://doi.org/10.1145/1869459.1869497.

[9] E. Kmett, *Bound*, Accessed: 2023-01-21, Dec. 2015. [Online]. Available: %7Bhttps://www.schoolofhaskell.com/user/edwardk/bound%7D.

[10] F. Pfenning and C. Elliott, "Higher-order abstract syntax," in *Proceedings of the ACM SIGPLAN'88 Conference on Programming Language Design and Implementation (PLDI), Atlanta, Georgia, USA, June 22-24, 1988*, R. L. Wexelblat, Ed., ACM, 1988, pp. 199–208. DOI: 10.1145/53990.54010.

[11] A. Chlipala, "Parametric higher-order abstract syntax for mechanized semantics," in *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming, ICFP 2008*, ser. SIGPLAN Notices, vol. 43, ACM, 2008, pp. 143–156. DOI: 10.1145/1411203.1411226. [Online]. Available: https://doi.org/10.1145/1411203.1411226.

[12] G. Washburn and S. Weirich, "Boxes go bananas: Encoding higher-order abstract syntax with parametric polymorphism," *Journal of Functional Programming*, vol. 18, no. 1, pp. 87–140, 2008. DOI: 10.1017/S0956796807006557. [Online]. Available: https://doi.org/10.1017/S0956796807006557.

[13] S. Peyton Jones and S. Marlow, "Secrets of the glasgow haskell compiler inliner," *Journal of Functional Programming*, vol. 12, pp. 393–434, Jul. 2002. [Online]. Available: https://www.microsoft.com/en-us/research/publication/secrets-of-the-glasgow-haskell-compiler-inliner/.

[14] M. J. Gabbay and A. M. Pitts, "A new approach to abstract syntax with variable binding," *Formal Aspects of Computing*, vol. 13, no. 3-5, pp. 341–363, 2002. DOI: 10.1007/s001650200016.

[15] M. R. Shinwell, A. M. Pitts, and M. J. Gabbay, "Freshml: Programming with binders made simple," in *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming, ICFP 2003*, ACM, 2003, pp. 263–274. DOI: 10.1145/944705.944729.

[16] C. Urban, "Nominal techniques in isabelle/hol," *Journal of Automated Reasoning*, vol. 40, no. 4, pp. 327–356, 2008. DOI: 10.1007/s10817-008-9097-2.

[17] R. S. Bird and R. Paterson, "De bruijn notation as a nested datatype," *J. Funct. Program.*, vol. 9, no. 1, pp. 77–91, Jan. 1999, ISSN: 0956-7968.

DOI: 10.1017/S0956796899003366. [Online]. Available: https://doi.org/10.1017/S0956796899003366.

[18] A. Charguéraud, "The locally nameless representation," *J. Autom. Reason.*, vol. 49, no. 3, pp. 363–408, 2012. DOI: 10.1007/S10817-011-9225-2. [Online]. Available: https://doi.org/10.1007/s10817-011-9225-2.

[19] C. McBride, "Everybody's got to be somewhere," in *Proceedings of the 7th Workshop on Mathematically Structured Functional Programming, MSFP@FSCD 2018, Oxford, UK, 8th July 2018*, R. Atkey and S. Lindley, Eds., ser. EPTCS, vol. 275, 2018, pp. 53–69. DOI: 10.4204/EPTCS.275.6. [Online]. Available: https://doi.org/10.4204/EPTCS.275.6.

[20] H. (Barendregt and E. Barendsen, "Introduction to lambda calculus," *Nieuw Archief voor Wiskunde*, vol. 4, pp. 337–372, Jan. 1984.

[21] M. Noonan, "Ghosts of departed proofs (functional pearl)," in *Proceedings of the 11th ACM SIGPLAN International Symposium on Haskell*, ser. Haskell 2018, ACM, 2018, pp. 119–131. DOI: 10.1145/3242744.3242755. [Online]. Available: https://doi.org/10.1145/3242744.3242755.

[22] C. Okasaki and A. Gill, "Fast mergeable integer maps," in *Workshop on ML*, 1998, pp. 77–86.

[23] M. Fiore and C.-K. Hur, "Second-order equational logic (extended abstract)," in *Computer Science Logic, 24th International Workshop, CSL 2010*, ser. Lecture Notes in Computer Science, vol. 6247, Springer, 2010, pp. 320–335. DOI: 10.1007/978-3-642-15205-4_26. [Online]. Available: https://doi.org/10.1007/978-3-642-15205-4_26.

[24] M. Fiore and D. Szamozvancev, "Formal metatheory of second-order abstract syntax," *Proceedings of the ACM on Programming Languages*, vol. 6,

no. POPL, 2022. DOI: 10.1145/3498715. [Online]. Available: https://doi.org/10.1145/3498715.

[25] W. Swierstra, "Data types à la carte," *Journal of Functional Programming*, vol. 18, no. 4, pp. 423–436, 2008. DOI: 10.1017/S0956796808006758. [Online]. Available: https://doi.org/10.1017/S0956796808006758.

[26] T. Altenkirch, J. Chapman, and T. Uustalu, "Monads need not be endofunctors," *Logical Methods in Computer Science*, vol. 11, no. 1, 2015. DOI: 10.2168/LMCS-11(1:3)2015. [Online]. Available: https://doi.org/10.2168/LMCS-11(1:3)2015.

[27] T. Sheard and S. Peyton Jones, "Template metaprogramming for Haskell," in *Proceedings of the 2002 ACM SIGPLAN Workshop on Haskell*, ser. Haskell 2002, ACM, 2002, pp. 1–16. DOI: 10.1145/581690.581691. [Online]. Available: https://doi.org/10.1145/581690.581691.

[28] M. Forsberg and A. Ranta, "Demonstration abstract : Bnf converter," 2004. [Online]. Available: https://api.semanticscholar.org/CorpusID:9335858.

[29] E. Meijer, M. Fokkinga, and R. Paterson, "Functional programming with bananas, lenses, envelopes and barbed wire," in *Functional Programming Languages and Computer Architecture, FPCA 1991*, ser. Lecture Notes in Computer Science, vol. 523, Springer, 1991, pp. 124–144. DOI: 10.1007/3540543961_7.

[30] R. Lämmel and S. Peyton Jones, "Scrap your boilerplate: A practical design pattern for generic programming," in *Proceedings of the 2003 ACM SIGPLAN International Workshop on Types in Languages Design and Implementation, TLDI 2003*, ACM, 2003, pp. 26–37. DOI: 10.1145/604174.604179.